QALight

Frontend-розробка

Заняття 10. Чуйний дизайн

Викладач курсу: Іван Ніконов, ТОВ «Тренінг-центр КьюЕйЛайт»

Порядок денний



Медіа-запити. Область перегляду браузера. Медіа-функції. Контрольні точки. Функція clamp.

CSS-реалізація чуйного дизайну сайту.

Адаптивний вебдизайн

Медіа-запити. Область перегляду браузера. Медіа-функції. Контрольні точки. Функція clamp.

Адаптивний вебдизайн

Теорія: Адаптивний вебдизайн

Чуйний (адаптивний) вебдизайн (**Responsive web design RWD**) - це підхід у веброзробці, завдяки якому сайт автоматично підлаштовується під будь-які розміри екранів: від мобільних телефонів до великих моніторів.

Попередник адаптивного дизайну: **дизайн мобільних веб-сайтів**.

До того, як адаптивний веб-дизайн став стандартним підходом для роботи веб-сайтів на різних типах пристроїв, веб-розробники говорили про мобільний веб-дизайн, розробку мобільних веб-сайтів або іноді про дизайн, адаптований для мобільних пристроїв. Це по суті те саме, що й адаптивний веб-дизайн - цілі полягають у тому, щоб переконатися, що веб-сайти добре працюють на пристроях з різними фізичними атрибутами (розмір екрана, роздільна здатність) з точки зору макета, контенту (тексту та медіа) та продуктивності.

Сучасні технології набагато краще підходять для створення адаптивного досвіду. Наприклад, адаптивні технології зображень/медіа тепер дозволяють показувати відповідні медіа на різних пристроях без необхідності покладатися на такі методи, як серверне сканування.

Метатег області перегляду

Теорія: Адаптивний вебдизайн

```
<!-- відображення контенту на всю ширину екрана пристрою -->
<meta name="viewport" content="width=device-width, initial-scale=1>
```

- initial-scale=1 означає, що розмеі сторінки дорівнює 100% величини області перегляду.
- **width**=**device-width** означає, що ширина сторінки дорівнює 100% ширини вікна браузера, отже її не потрібно масштабувати.

Перед написанням CSS необхідно додати **meta**-тег viewport у блок <head> HTML-документа. Без нього мобільні браузери будуть просто масштабувати десктопну версію сайту, роблячи текст дрібним. viewport повідомляє мобільним браузерам, що слід встановити ширину області перегляду на ширину пристрою та масштабувати документ до 100% від його запланованого розміру, що відображає документ у розмірі, оптимізованому для мобільних пристроїв.

Метатег viewport існує тому, що коли смартфони тільки з'явилися, більшість сайтів не були оптимізовані для мобільних пристроїв. Тому мобільний браузер встановлював ширину області перегляду на 980 пікселів, відображав сторінку з цією шириною та відображав результат як зменшену версію макета робочого столу.

Значення за замовчуванням мобільного пристрою

Теорія: Адаптивний вебдизайн

Тег viewport можна використовувати для контролю того, на яку величину користувач може збільшувати чи зменшувати ширину сторінки:

```
<meta name="viewport" content="width=device-width, maximum-scale=3, minimum-scale=0.5">
```

Встановлюючи **width=device-width,** ви замінюєте значення за замовчуванням мобільного пристрою, наприклад, значення за замовчуванням iPhone = 980px, фактичною шириною пристрою.

Без цього ваш адаптивний дизайн з точками зупинки та медіа-запитами може не працювати належним чином у мобільних браузерах.

Якщо у вас вузький макет екрана, який запускається з шириною області перегляду 480px або менше, але пристрій показує ширину 980px, користувач не побачить ваш вузький макет екрана.

Медіа-запити

Теорія: Медіа-запити

Медіа-запити дозволяють нам виконувати серію тестів (наприклад, чи екран користувача більший за певну ширину або роздільну здатність) та вибірково застосовувати CSS для стилізації сторінки відповідно до потреб користувача.

Можна задавати медіа-запити, щоб перевірити:

- ширину та висоту вікна перегляду
- ширину і висоту пристрою
- орієнтацію екрана (альбомна чи книжкова)
- розширення екрану

Маючи можливість перевірити цю інформацію, можна створити правила, які відображатимуть макет у спосіб, який найкраще підходить для пристрою, який використовується.

Директива @media

Теорія: Медіа-запити

Для створення правила CSS використовується директива **@media**, що виконується лише за умови істинності вказаної властивості.

Наприклад, якщо вікно браузера пристрою має розмір 600 пікселів або менше, встановіть колір фону на orange:

```
@media only screen and (max-width: 600px) {
  body {
    background-color: orange;
  }
}
```

at-правило @media може бути розміщено на верхньому рівні вашого коду або вкладено всередину будь-якого іншого умовного групового at-правила.

Кілька медіа-запитів

Ви можете додати кілька медіа-запитів у таблицю стилів, налаштувавши весь макет або його частини, щоб найкраще відповідати різним розмірам екрана. Точки, в яких вводиться медіа-запит і змінюється макет, називаються точками зупинки.

Наприклад, наступний медіа-запит перевіряє, чи відображається поточна веб-сторінка як екранний медіафайл (тобто не друкований документ), і чи ширина області перегляду становить щонайменше 80rem. \

Правило застосовуватиметься лише за умови, що ці дві умови є істинними.

```
@media screen and (width >= 80rem) {
.container {
margin: 1em 2em;
}
}
```

Типи медіа

Теорія: Типи медіа

Типи медіа дозволяють завантажувати CSS на основі типу пристрою.

Доступні такі типи медіа:

- all - Підходить для всіх пристроїв.
- print - Призначено для сторінкового матеріалу та документів, що переглядаються на екрані в режимі попереднього перегляду друку.
- screen - Призначено для відображення на екрані (комп'ютери, планшети тощо)
- speech - Призначено для програм зчитування з екрана

<media-type> описує загальну категорію пристрою. За винятком випадків використання єдиного логічного оператора, тип медіа є необов'язковим, а тип all мається на увазі.

screen є найбільш часто використовуваним типом.

Контрольні точки (Breakpoints)

Теорія: Медіа-запити

Існує безліч екранів і пристроїв різної висоти та ширини, тому створити точну точку розриву (breakpoint) для кожного пристрою досить важко.

Адаптивні макети використовцють шість **контрольних** (**Breakpoints**) точок відповідно до ширини екрана: 320, 480, 760, 960, 1200, 1600. Для простого процесу кодування адаптивних макетів створюють медіа-запити, націлені на пристрої на основі діапазонів розмірів екрана:

```
/* Extra small devices (phones, less than 600px wide) */
@media only screen and (max-width: 600px) {...}
/* Small devices (portrait tablets & large phones, 600px and up) */
@media only screen and (min-width: 600px) {...}
/* Medium devices (landscape tablets, 768px and up) */
@media only screen and (min-width: 768px) {...}
/* Large devices (laptops/desktops, 992px and up) */
@media only screen and (min-width: 992px) {...}
/* Extra large devices (large laptops & desktops, 1200px and up) */
@media only screen and (min-width: 1200px) {...}
```

Підхід Mobile First

Теорія: Медіа-запити

Найчастіше використовують підхід Mobile First

Запити потрібно розміщувати **лише таким порядком**.

```
/* Custom, iPhone Retina */
@media only screen and (min-width : 320px) {}
/* Extra Small Devices, Phones */
@media only screen and (min-width : 480px) {}
/* Small Devices, Tablets */
@media only screen and (min-width : 768px) {}
/* Medium Devices, Desktops */
@media only screen and (min-width : 992px) {}
/* Large Devices, Wide Screens */
@media only screen and (min-width : 1200px) {}
```

Розширення для великих екранів

Теорія: Медіа-запити

Запити потрібно розміщувати **лише таким порядком**.

```
/* Large Devices, Wide Screens */
@media only screen and (max-width : 1200px) {}
/* Medium Devices, Desktops */
@media only screen and (max-width : 992px) {}
/* Small Devices, Tablets */
@media only screen and (max-width : 768px) {}
/* Extra Small Devices, Phones */
@media only screen and (max-width : 480px) {}
/* Custom, iPhone Retina */
@media only screen and (max-width : 320px) {}
```

Запити у вказаних діапазонах

Теорія: Медіа-запити

Медіа запити у вказаних діапазонах.

```
@media (max-width: 767px) {     /* стили для xs-устройств */   }
@media (min-width: 768px) and (max-width: 991px) {  /* стили для sm-устройств */   }
@media (min-width: 991px) and (max-width: 1199px) {  /* стили для md-устройств */  }
@media (min-width: 1200px) {  /* стили для lg-устройств */   }
```

Створення простого одноколонкового макета

Теорія: Медіа-запити

Поширений підхід до використання медіазапитів полягає у створенні простого одноколонкового макета для вузьких екранів пристроїв (наприклад, мобільних телефонів), а потім перевірці на ширші екрани та реалізації багатоколонкового макета, коли ви знаєте, що ширини екрана достатньо для його обробки.

```
/* Базові стилі для мобільних телефонів (екран < 768px) */
body {
 font-size: 14px;
 padding: 10px;
}
```

Дизайн, орієнтований на мобільні пристрої

Теорія: Медіа-запити

Проєктування для мобільних пристроїв відоме як дизайн, орієнтований на мобільні пристрої.

```
/* Планшети (від 768px і більше) */
@media (min-width: 768px) {
 body {
   font-size: 16px;
   padding: 20px;
 }
}

/* Ноутбуки та комп'ютери (від 1024px і більше) */
@media (min-width: 1024px) {
 body {
   font-size: 18px;
   max-width: 1200px;
   margin: 0 auto; /* Центрування контенту */
 }
}
```

Відносні одиниці виміру

Теорія: Медіа-запити

Якщо використовуються точки зупинки, найкращі практики рекомендують визначати точки зупинки медіазапитів з відносними одиницями, а не з абсолютними розмірами окремого пристрою.

```
html {
 font-size: 16px; /* Базовий розмір */
}

h1 {
 font-size: 2.5rem; /* 16px * 2.5 = 40px */
}
.sidebar {
 width: 25%; /* Займатиме чверть ширини батьківського блоку */
}
```

Замість фіксованих пікселів (px) використовуйте відносні одиниці:% — відсотки від батьківського елемента.vw / vh — відсотки від ширини/висоти всього екрана (viewport).rem — одиниця, що базується на розмірі шрифту кореневого елемента (<html>). Полегшує масштабування всього сайту.

Медіа-функції

Практика: Медіа-функції

Медіа-функції (**<media feature>**) описує певні характеристики користувацького агента, пристрою виводу або середовища.

Вирази медіа-функцій перевіряють їх наявність, значення або діапазон значень і є абсолютно необов'язковими.

Кожен вираз медіа-функції має бути взято в дужки.

Логічні оператори

Практика: логічні оператори

Для створення більш складного запиту використовують логічні оператори not, and, і only.

Оператор **and** - використовуємо та об’єднуємо кілька медіа-функцій разом в один медіа-запит. Кожна функція має бути істинною, щоб запит повернув true. Оператор and також використовується для з’єднання мультимедійних функцій із типами медіа.

Оператор **not** використовується для заперечення медіа-запиту. Заперечення медіа-запиту буде істиною, якщо в іншому випадку запит повертає false. Якщо використовується not, також потрібно вказати тип медіа.

Оператор **only** використовується для застосування стилю, лише якщо істині відповідає весь запит. Це корисно для запобігання застосуванню вибраних стилів у старих браузерах. Якщо використовується only, ви також повинні вказати тип медіа.

Оператор **, (comma)** дозволяє об’єднати кілька медіа-запитів в одне правило, розділивши їх комами. Якщо будь-який із запитів у списку відповідає дійсності, весь оператор повертає true.

Оператор and

Практика: логічні оператори

Оператор and - використовуємо та об’єднуємо кілька медіа-функцій разом в один медіа-запит. Кожна функція має бути істинною, щоб запит повернув true.

```
@media only screen and (min-width: 768px) {
    p {padding: 30px;}
  }
```

Запит поверне **true**, якщо тип медіафайлу відповідає типу пристрою, на якому відображається сторінка, і якщо всі вирази в медіа-запиті відповідають дійсності. Коли медіа-запит повертає **true**, будуть застосовані відповідні правила стилю.

Тут правило застосовується, коли у користувача є екранний пристрій, ширина якого становить від 600 до 768 пікселів, а орієнтація - альбомна:

```
@media screen and (max-width: 768px) and (min-width: 600px) and (orientation: landscape) {
  /* ... */
}
```

Оператор and також використовується для з’єднання мультимедійних функцій із типами медіа.

```
@media only screen and (min-height: 768px) and (orientation: landscape) {
  p {padding: 30px;}
}
```

Оператор comma

Практика: логічні оператори

Оператор , (comma) дозволяє об’єднати кілька медіа-запитів в одне правило, розділивши їх комами. Якщо будь-який із запитів у списку відповідає дійсності, весь оператор повертає true.

The OR syntax

```
@media only screen and (min-width: 768px), (min-resolution: 150dpi) {
  p {padding: 30px;}
}
```

Оператор not

Практика: логічні оператори

Оператор not використовується для заперечення медіа-запиту. Заперечення медіа-запиту буде істиною, якщо в іншому випадку запит повертає false. Якщо використовується not, також потрібно вказати тип медіа.

```
@media all and (not handheld) { ... }

@media not all and (color) { ... }

@media not (all and (color)) { ... }

@media (not all) and (color) { ... }
```

Оператор only

Практика: логічні оператори

Оператор only використовується для застосування стилю, лише якщо істині відповідає весь запит. Це корисно для запобігання застосуванню вибраних стилів у старих браузерах. Якщо використовується only, ви також повинні вказати тип медіа.

```
@media only all and (not handheld) { ... }
```

Альбомний або портретний режим

Практика: Медіа-запити

Визначає, що **пристрій знаходиться в альбомному режимі (ширина більше за висоту) або портретному (ширина меньше за висоту)**.

```
/* landscape - режим, ширина viewport більша за висоту */
    @media (orientation: landscape) {
      #background-image { background: url(image1.png) no-repeat; }
    }
/* portrait - висота viewport больша за ширину */
    @media (orientation: portrait) {
      #background-image { background: url(image2.png) no-repeat; }
    }
```

Точна мінімальна ширина

Практика: Медіа-запити

Точна мінімальна ширина може вказуватись у відсотках, em, rem чи px:
@media screen and (min-width: 50em) { /* стили */}

Директива @media повідомляє браузеру про початок медіа-запиту, компонент **screen** сповіщає браузер, що правила мають бути застосовані до всіх типів екранів, компонент and (**min-width: 50em**) сповіщає браузер, що правила мають діяти для всіх вікон перегляду, чия ширина перевищує 50 em.

```
@media (min-width: 544px) { /* Стили CSS ... */ }
```

Точна мінімальна ширина може вказуватись у відсотках, em, rem чи px:

Точна максимальна ширина

Практика: Медіа-запити

max-width - вказує максимальну робочу область браузера.

/* Використовуйте функцію медіа max-width, щоб застосовувати стилі до об’єктів шириною менше певної величини: */

```
@media (max-width: 400px) {
 /* Styles for viewports narrower than 400 pixels. */
}
```

Цей медіа-запит

```
@media (max-width: 400px) {
 /* Styles for viewports narrower than 400 pixels. */
}
```

також можна написати за допомогою функції медіа ширини та оператора «менше або дорівнює».

```
@media (width <= 400px) {
 /* Styles for viewports narrower than 400 pixels. */
}
```

Бажане значення між двома точками зупинки

Практика: Медіа-запити

```
 /* Бажане значення між мінімальною та максимальною межею */

@media (min-width: [breakpoint-min]) {
 .fluid {
        font-size: calc([value-min] + ([value-max] - [value-min]) * ((100vw - [breakpoint-min]) / ([breakpoint-max]
- [breakpoint-min])));
 }
}

/* медіа-запит обробляє плавну поведінку між двома точками зупинки 568px і 768px */

@media screen and (min-width: 568px) {
 .fluid { font-size: calc(32px + 16 * ((100vw - 568px) / (768 - 568)); }
}
```

Гнучкі сітки

Практика: Гнучкі сітки

Використовуйте сучасні CSS-моделі макетування, які автоматично розподіляють простір.

CSS Flexbox (ідеально для лінійних структур: меню, списки)

```
.nav-container {
 display: flex;
 flex-direction: column; /* На мобільних по вертикалі */
 gap: 10px;
}

@media (min-width: 768px) {
 .nav-container {
   flex-direction: row; /* На планшетах/ПК по горизонталі */
   justify-content: space-between;
 }
```

Адаптивні зображення/медіафайли

Практика: Адаптивні зображення/медіафайли

Гумові зображення та медіафайли
Щоб картинки не вилазили за межі екрана на мобільних пристроях, їм потрібно задати максимальну ширину:

```
img, video {
 max-width: 100%;
 height: auto; /* Зберігає пропорції */
 display: block;
}
```

Приклад CSS-реалізації для навігаційного меню

Практика: підхід Mobile First

Приклад CSS-реалізації для навігаційного меню та сітки карток.
Ми використаємо підхід Mobile First: спочатку напишемо стилі для смартфонів, а потім за допомогою медіазапитів (@media) адаптуємо їх під великі екрани.

```
<header class="site-header">
 <div class="logo">МійСайт</div>
 <nav class="main-nav">
   <a href="#">Головна</a>
   <a href="#">Послуги</a>
   <a href="#">Портфоліо</a>
   <a href="#">Контакти</a>
 </nav>
</header>
```

Сітка карток

Практика: підхід Mobile First

```
<main class="content-container">
 <section class="cards-grid">
  <div class="card">
   <div class="card-image"></div>
   <div class="card-body">
    <h3>Картка 1</h3>
    <p>Це текст опису для першої картки. Він адаптується під розмір.</p>
   </div>
  </div>
  <div class="card">
   <div class="card-image"></div>
   <div class="card-body">
    <h3>Картка 2</h3>
    <p>Це текст опису для другої картки. Він адаптується під розмір.</p>
   </div>
  </div>
 </section>
</main>
```

Адаптивний CSS

Практика: підхід Mobile First

```css
/* --- НАВІГАЦІЯ (Мобільна версія: посилання одне під одним) --- */
.site-header {
 background-color: #ffffff;
 padding: 15px;
 box-shadow: 0 2px 5px rgba(0,0,0,0.05);
 display: flex;
 flex-direction: column; /* Елементи шикуються у стовпчик */
 gap: 15px;
 align-items: center;
}

.logo {
 font-size: 1.5rem;
 font-weight: bold;
 color: #0070f3;
}
```

НАВІГАЦІЯ (Мобільна версія)

Практика: підхід Mobile First

```css
.main-nav {
 display: flex;
 flex-direction: column; /* Посилання меню йдуть одне під одним */
 gap: 10px;
 width: 100%;
 text-align: center;
}

.main-nav a {
 text-decoration: none;
 color: #555;
 padding: 8px;
 background-color: #f0f0f0;
 border-radius: 5px;
 font-weight: 500;
}
```

СІТКА КАРТОК (Мобільна версія)

Практика: підхід Mobile First

```css
/* --- СІТКА КАРТОК (Мобільна версія: 1 картка в ряд) --- */
.content-container {
 padding: 20px;
 max-width: 1200px;
 margin: 0 auto;
}

.cards-grid {
 display: grid;
 /* auto-fit + minmax роблять магію: якщо екран зовсім малий,
    картка займе 100% ширини, але не стане меншою за 280px */
 grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
 gap: 20px;
}
```

СІТКА КАРТОК (Мобільна версія)

Практика: підхід Mobile First

```
.card {
 background: #ffffff;
 border-radius: 8px;
 overflow: hidden;
 box-shadow: 0 4px 6px rgba(0,0,0,0.05);
}

.card-image {
 height: 180px;
 background-color: #ddd; /* Заглушка замість картинки */
}
.card-body {
 padding: 15px;
}
.card-body h3 {
 margin-bottom: 10px;
}
```

АДАПТАЦІЯ ДЛЯ ПЛАНШЕТІВ ТА ПК

Практика: підхід Mobile First

```
/* --- СІТКА КАРТОК (Мобільна версія: 1 картка в ряд) --- */
.content-container {
 padding: 20px;
 max-width: 1200px;
 margin: 0 auto;
}

.cards-grid {
 display: grid;
 /* auto-fit + minmax роблять магію: якщо екран зовсім малий,
    картка займе 100% ширини, але не стане меншою за 280px */
 grid-template-columns: repeat(auto-fit, minmax(280px, 1fr));
 gap: 20px;
}
```

АДАПТАЦІЯ ДЛЯ ПЛАНШЕТІВ ТА ПК

Практика: підхід Mobile First

```
.card {
 background: #ffffff;
 border-radius: 8px;
 overflow: hidden;
 box-shadow: 0 4px 6px rgba(0,0,0,0.05);
}
.card-image {
 height: 180px;
 background-color: #ddd; /* Заглушка замість картинки */
}
.card-body {
 padding: 15px;
}
.card-body h3 {
 margin-bottom: 10px;
}
```

Адаптивна типографіка

Практика: Адаптивна типографіка

Адаптивна типографіка описує зміну розмірів шрифтів у медіа-запитах або використання одиниць області перегляду для відображення меншої або більшої площі екрана.

```
 /* Максимальне значення */
@media (min-width: [breakpoint-max]) {
 .fluid {
 font-size: [value-max]
 }
}
```

```
/* Fixed maximum value above the maximum breakpoint */
@media screen and (min-width: 768px) {
    .fluid {
        font-size: 48px;
    }
}
```

Використання функції clamp()

Практика: спосіб реалізації плинної типографіки

Використання функції clamp() - це найсучасніший спосіб реалізації плинної типографіки (Fluid Typography). Замість того, щоб різко перемикати розміри тексту в медіазапитах (@media), шрифт буде плавно й безперервно масштабуватися разом із кожним пікселем зміни ширини екрана.

Функція **clamp** приймає три значення - **мінімальну межу**, **бажане значення** та **максимальну межу**, і фіксує поточне значення між цими межами.
Бажане значення використовується для визначення значення між межею. Бажане значення зазвичай включає одиниці viewport, відсотки або інші відносні одиниці для досягнення ефекту плавноств. Функція може приймати навіть математичні функції та вирази, а також значення з функції attr.

clamp([value-min], [value-preferred], [value-max]);

Цю функцію можна застосувати до будь-якого атрибута, який приймає дійсний тип значення, як-от довжина, частота, час, кут, відсоток, число та інші, тому її можна використовувати не лише для типографіки та розміру.

Як працює формула clamp()

Практика: спосіб реалізації плинної типографіки

```
h1 {
 font-size: clamp(1.8rem, 4vw + 1rem, 3.5rem);
}
```

- 1.8rem (Мінімум): Найменший розмір шрифту. Меншим за це значення він не стане ні за яких умов (ідеально для смартфонів).
- 3.5rem (Максимум): Найбільший розмір шрифту. Більшим за це значення він не виросте (ідеально для UltraWide моніторів).
- 4vw + 1rem (Оптимальне динамічне значення): Розрахункова величина. 4vw означає 4% від ширини екрана. Додавання 1rem є обов'язковим для збереження доступності (щоб текст масштабувався, якщо користувач змінить базовий розмір шрифту в налаштуваннях свого браузера).

Навігаційне меню

Практика: спосіб реалізації плинної типографіки

```
/* Навігаційне меню (Логотип) */
.logo {
 /* Плавно змінюється від ~22px до ~32px */
 font-size: clamp(1.375rem, 1.5vw + 1rem, 2rem);
}

/* Головні заголовки секцій */
.site-section h2 {
 /* Плавно змінюється від ~28px до ~48px */
 font-size: clamp(1.75rem, 3vw + 1rem, 3rem);
 margin-bottom: 20px;
}
/* Заголовки всередині карток */
.card-body h3 {
 /* Плавно змінюється від ~18px до ~24px */
 font-size: clamp(1.125rem, 1vw + 0.9rem, 1.5rem);
 margin-bottom: 10px;
}
```

ВПРАВИ



Вправи до цього заняття знаходяться за посиланням
https://github.com/coachjanus/frontend-dev

Посилання на довідники



Vs Code : https://code.visualstudio.com/

Webstorm: https://www.jetbrains.com/webstorm/

Web technology For developers https://developer.mozilla.org/en-US/docs/Web

CSS: https://developer.mozilla.org/en-US/docs/Web/CSS

https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/CSS_layout/Responsive_Design

Консорціум Всесвітнього павутиння https://www.w3.org/

Дякую за увагу!

До наступної зустрічі!

QALight

www.qalight.ua
info@qalight.ua